Slide 1 — Week 4

Analytical and Non-Relational Data Systems

Topics:

- OLTP vs OLAP; data warehouses
- Star schema: fact and dimension tables
- Grain, surrogate keys, SCD Type 2
- Relational vs object-oriented and document models
- Modern data architectures

Slide 2 — Operational vs Analytical Databases

Two types of databases exist in most organizations.

Operational databases (OLTP)

Analytical databases (OLAP)

They serve different purposes.

Slide 3 — OLTP Systems

OLTP = Online Transaction Processing

Examples:

- order processing
- banking systems
- customer management systems

Characteristics

- many small transactions
- normalized schemas

Slide 4 — OLAP Systems

OLAP = Online Analytical Processing

Examples

- sales analysis
- customer analytics
- forecasting

Characteristics

- large queries
- aggregations
- historical data

Slide 5 — Example OLTP Schema

Tables

customers

orders

order_items

products

payments

This schema is optimized for transactions.

Slide 6 — Analytical Questions

Example business questions

- Which products generate the most revenue?
- Which countries produce the most sales?
- What are monthly sales trends?

OLAP systems answer these efficiently.

Slide 7 — Data Warehouses

A **data warehouse** stores data optimized for analytics.

Characteristics

- historical data
- integrated data
- optimized queries

Slide 8 — Star Schema

Most common warehouse design.

Structure

Central **fact table**

Connected **dimension tables**

Slide 9 — Fact Table

Fact tables store measurable metrics.

Example columns

Keys to the dimensions: customer_key, product_key, date_key

Measures: quantity, sales_amount

Slide 10 — Dimension Tables

Dimension tables provide context.

Examples

dim_customer

dim_product

dim_date

dim_store

Slide 11 — Example Star Schema

Fact table

fact_sales

Dimension tables

dim_customer

dim_product

dim_date

Slide 12 — Grain: The First Decision

The **grain** is one sentence saying what a single fact row means.

> "One row per sale line."

Write it before any DDL. It decides:

- which dimensions can attach
- which measures can be summed
- what COUNT(*) means

A wrong grain makes every aggregate wrong — and the numbers still look plausible.

Slide 13 — Surrogate Keys and Unknown Members

Surrogate keys — the fact stores an integer the warehouse controls (product_key), not the source system's business key (product_id).

This makes history (SCD Type 2) possible and survives source renumbering.

Unknown member — every dimension has a row at key -1.
An unmatched fact maps to -1, never NULL — a NULL key silently drops the row from every inner join.

Slide 14 — Example Fact Table

```
CREATE TABLE fact_sales (
  customer_key INT,   -- surrogate keys into the dimensions
  product_key  INT,
  date_key     INT,
  quantity     INT,
  revenue      DECIMAL(10,2)
);
```

Grain: one row per sale line.

Slide 15 — Example Dimension Table

```sql
CREATE TABLE dim_product (
  product_key  INT PRIMARY KEY,   -- surrogate key
  product_id   VARCHAR,           -- business key from the source
  product_name VARCHAR(100),
  category     VARCHAR(100)
);
```

Slide 16 — Slowly Changing Dimensions

A store moves to a new region. Do last year's sales move with it?

Type 1 — overwrite; history lost (corrections)

Type 2 — new row per version, with valid_from / valid_to and a current flag

Type 3 — keep a "previous value" column

Type 2 is the default for anything you report on historically.
Test it: one current row per key, no overlaps, no gaps.

Slide 17 — Benefits of Star Schema

Advantages

- simple queries
- fast aggregations
- easy analytics

Widely used in business intelligence systems.

Slide 18 — Example OLAP Query

```sql
SELECT dp.category,
       SUM(fs.revenue) AS revenue
FROM fact_sales fs
JOIN dim_product dp
  ON fs.product_key = dp.product_key
GROUP BY dp.category;
```

This query calculates revenue by product category.

Slide 19 — Data Lakes

Data lakes store raw data.

Examples

- logs
- CSV files
- JSON data

They support large-scale analytics.

Slide 20 — Lakehouse Architecture

Lakehouse combines

Data warehouse + Data lake

Benefits

- flexible storage
- strong analytics

Slide 21 — Medallion Architecture

Modern lakehouse systems use three layers.

Bronze — raw data

Silver — cleaned data

Gold — analytics-ready data

Slide 22 — Analytical Data Flow

OLTP database

↓

ETL / Data pipeline

↓

Data warehouse

↓

Analytics dashboards

Slide 23 — Relational vs Object-Oriented Databases

An application holds an **object**; a normalized database holds **tables**.

Translating between them is the **impedance mismatch**.

Object-oriented databases (1980s–90s) stored objects directly:

- object identity instead of keys
- references instead of joins
- inheritance and methods stored with the data

Slide 24 — Why Relational Won the Mainstream

- no query language as powerful as SQL
- navigation fixes the access path; SQL lets the optimizer choose
- schema changes were painful
- weak at analytics (sums across many objects)
- relational vendors added objects themselves (SQL:1999 object-relational types)

Slide 25 — The Document Model

Document stores (MongoDB, Couchbase, DynamoDB) kept the core idea:

- store a whole aggregate — e.g. one order — as one JSON document
- no join to reassemble it
- flexible schema: documents may differ

What they gave up, at first: joins and multi-document transactions.

Slide 26 — Three Ways to Model One Order

Relational — orders, order_lines, products; reassemble with joins

Object-relational — one row; lines as LIST(STRUCT(...)); query across with UNNEST

Document — one JSON column; path expressions, no schema guarantees

All three run in DuckDB.

The trade is always **locality vs flexibility of access**.

Slide 27 — When Documents Change: Schema Drift

A document model accepts any shape — so the shape **drifts**:

- a field is **added** — `geo` appears in v2 of the claims app
- a field is **renamed** — `device.app` → `device.app_version`
- a field **changes type** — `speed_kph` arrives as `62`, then as `"48"`
- a field is **missing** — `photos` only on some claims

Nothing rejects the change; your queries silently return `NULL` or wrong totals.

Today, in depth: Week 1 notes, Part 3 — **Part B: Semi-Structured Data**.
Using the Harborview Mutual claims case: profiling drift with SQL, shredding JSON into tables, design patterns, and the EAV trap.

Slide 28 — Choosing a Model

Dominant pattern	Prefer
Read/write one aggregate by key	document or nested
Ad-hoc queries across entities	relational
Strong cross-entity rules	relational
Schema varies per record	document
Analytics and aggregation	relational / columnar

Often the answer is both: documents for the app, relational for the warehouse.

Slide 29 — Week 4 Labs

Each session has a core lab (**a**) and a challenge lab (**b**), started in the last ~40 minutes of class and submitted by 11:00 pm.

Session 7 — From OLTP to a star schema

- Lab 1a — Marigold Garden: grain, surrogate keys, fact and dimension tables, a date dimension
- Lab 1b — Northvale Grocers: conformed dimensions, SCD Type 2, as-of loading, additivity

Session 8 — Querying a warehouse; relational vs document models

- Lab 2a — Peakline Outdoor: drill-down, `ROLLUP`, window functions, top-N per group
- Lab 2b — Brightwater: one order as relational, nested, and document data (see Part 2)